ENASTIC
Ecole Nationale Supérieure des TIC

Projet de Fin d'Etudes

En vue de l'obtention du **Diplome de Licence en Informatique**

THEME

CyberSentinel-LLM :

Plateforme intelligente de detection d'anomalies
et d'assistance SOC basee sur le Machine Learning
et les Modeles de Langage

Presente par :

M. YAKHOUB MAHAMAT HAMDO

Directeur de memoire :

Dr. MOUAZ MICKAIL

Enseignant chercheur en IA

Annee Universitaire : 2025–2026

RÉSUMÉ

CyberSentinel-LLM est une plateforme pédagogique de détection d'anomalies et d'assistance SOC combinant Machine Learning, modèle de langage local augmenté par RAG, et interface vocale bidirectionnelle.

Le système ingère des logs réseau et les analyse via un pipeline IsolationForest/LOF avec fallback heuristique. Chaque alerte suspecte est enrichie par un module RAG (ChromaDB + 15 techniques MITRE ATT&CK) avant d'être soumise au LLM local Ollama/Llama 3.2, qui génère une explication structurée en JSON avec un score de confiance pondéré ($score_{ML} \times 0.4 + score_{RAG} \times 0.3 + score_{LLM} \times 0.3$). La plateforme intègre également une interface vocale complète : transcription audio via Faster-Whisper (STT) et synthèse vocale via Piper (TTS), le tout fonctionnant en local sur CPU sans GPU dédié.

Le prototype est déployable sur une machine modeste (Intel Core i7 11e génération, 16 GB RAM) et repose sur une stack open source : FastAPI, Next.js 14, SQLite, GeoLite2, ChromaDB et Ollama. Le principe Human-in-the-Loop est maintenu à tous les niveaux du système.

Mots-clés : IDS, Machine Learning, LLM, RAG, MITRE ATT&CK, ChromaDB, Ollama, STT, TTS, SOC, GeoIP, Human-in-the-Loop.

ABSTRACT

CyberSentinel-LLM is a pedagogical anomaly detection and SOC assistance platform combining Machine Learning, a RAG-augmented local language model, and a bidirectional voice interface.

The system ingests network logs and analyzes them through an IsolationForest/LOF pipeline with heuristic fallback. Each suspicious alert is enriched by a RAG module (ChromaDB + 15 MITRE ATT&CK techniques) before being submitted to the local LLM Ollama/Llama 3.2, which generates a structured JSON explanation with a weighted confidence score. The platform also integrates a complete voice interface : audio transcription via Faster-Whisper (STT) and speech synthesis via Piper (TTS), all running locally on CPU without a dedicated GPU.

The prototype is deployable on a modest machine (Intel Core i7 11th generation, 16 GB RAM) using an open-source stack : FastAPI, Next.js 14, SQLite, GeoLite2, ChromaDB and Ollama. The Human-in-the-Loop principle is maintained at all system levels.

Keywords : IDS, Machine Learning, LLM, RAG, MITRE ATT&CK, ChromaDB, Ollama, STT, TTS, SOC, GeoIP, Human-in-the-Loop.

AVANT-PROPOS

L'Ecole Nationale Superieure des Technologies de l'Information et de la Communication (ENASTIC) est un etablissement public d'enseignement, de recherche, d'innovation et de formation superieure dans les domaines des communications electroniques. Elle est creee par l'Ordonnance **N°005/PR/2015** du 02 mars 2015, dotee de la personnalite juridique et de l'autonomie financiere.

Placee sous la tutelle du Ministere des Postes et des Nouvelles Technologies de l'Information et de la Communication, et du Ministere de l'Enseignement Superieur, de la Recherche et de l'Innovation pour l'organisation academique et pedagogique, l'ENASTIC a pour missions de :

- ☞ Assurer la formation initiale et continue des cadres dans les domaines des Technologies de l'Information et de la Communication ;
- ☞ Entreprendre la recherche appliquee d'interet general, en vue de promouvoir l'innovation pour le developpement socio-economique du pays ;
- ☞ Former des jeunes a la culture scientifique afin de les rendre aptes a elaborer et a porter des projets structurants adosses aux TIC.

Pour completer leurs connaissances theoriques, les etudiants doivent preparer un Projet de Fin d'Etudes (PFE) afin de le presenter devant un jury de soutenance pour parachever leur cycle de Licence.

C'est dans cette optique que le present memoire vient marquer la fin de notre cursus en Informatique a l'ENASTIC, sur le theme : **« CYBERSENTINEL-LLM : PLATEFORME INTELLIGENTE DE DETECTION D'ANOMALIES ET D'ASSISTANCE SOC BASEE SUR LE MACHINE LEARNING ET LES MODELES DE LANGAGE »**, dans le cadre de l'annee universitaire 2025–2026.

Ce travail constitue une preuve de concept pedagogique combinant detection d'anomalies, assistance conversationnelle vocale par un modele de langage local augmente par une base de connaissances cybersecurite (RAG), afin d'aider les analystes SOC a interpreter et prioriser les alertes.

DEDICACE

Je dedie ce modeste travail a mes parents.

A mon defunt pere, qui a su me soutenir jusqu'a son dernier souffle —
il etait mon modele, et tout ce que j'entreprends est pour lui.

REMERCIEMENTS

Le present memoire est le resultat d'un travail effectue avec l'aide et le soutien de nombreuses personnes, a qui j'adresse mes sinceres remerciements.

Je tiens a exprimer ma profonde gratitude a :

- ☞ Mon Directeur de memoire, **Dr. MOUAZ MICKAIL**, Enseignant chercheur en Intelligence Artificielle, pour la qualite de son encadrement, sa disponibilite et ses precieux conseils tout au long de la realisation de ce projet ;
- ☞ L'equipe pedagogique de l'Ecole Nationale Superieure des Technologies de l'Information et de la Communication (ENASTIC), pour la qualite de la formation dispensee ;
- ☞ Ma famille et mes proches, pour leur soutien moral et leurs encouragements constants tout au long de ce cursus.

TABLE DES MATIÈRES

Résumé **I**

Abstract **II**

Avant-propos **III**

Dedicace **IV**

Remerciements **V**

Liste des figures **VIII**

Liste des tableaux **IX**

Liste des Abreviations **X**

Introduction Generale **1**

1 Etat de l'Art et Generalites **4**
Introduction 4
1.1 Presentation des IDS et IPS 4
1.2 Limites des approches classiques 4
1.3 Apport du Machine Learning en detection d'anomalies 5
1.4 Apport des LLM et principe Human-in-the-Loop 6
1.5 Retrieval-Augmented Generation (RAG) pour la cybersecurite 6
1.6 Interface vocale : STT et TTS pour les SOC 6
Conclusion 7

2 Analyse des Besoins et Specifications **8**
Introduction 8
2.1 Cas d'usage principaux 8
2.2 Exigences fonctionnelles 10
2.3 Exigences non fonctionnelles 10
Conclusion 10

3 Conception Logicielle et Architecture **11**
Introduction . . . 11
3.1 Architecture globale . . . 11
3.2 Arborescence du projet . . . 12
3.3 Description des composants backend . . . 13
3.4 Description du frontend . . . 13
3.5 Diagrammes UML . . . 14
3.5.1 Diagramme de classes . . . 14
3.5.2 Diagramme de sequence . . . 14
3.5.3 Diagramme d'activite . . . 14
Conclusion . . . 14

4 Implementation **17**
Introduction . . . 17
4.1 Endpoints principaux . . . 17
4.2 Pipeline Machine Learning . . . 17
4.3 Module RAG — Retrieval-Augmented Generation . . . 19
4.4 Module LLM avec score de confiance pondere . . . 19
4.5 Modules vocaux — STT et TTS . . . 20
4.6 SSE et communication temps reel . . . 21
4.7 Module de defense applicative . . . 21
Conclusion . . . 22

5 Tests, Validation et Resultats **23**
Introduction . . . 23
5.1 Scenarios de demonstration . . . 23
5.2 Mecanismes d'evaluation . . . 23
5.3 Resultats observes . . . 23
5.4 Tests automatises . . . 24
Conclusion . . . 24

Conclusion Generale et Perspectives **26**

Bibliographie **27**

Annexes **28**
Annexe A : Architecture globale . . . 28
Annexe B : Exemples de payloads API . . . 29
Annexe C : Scripts utilitaires . . . 30
Annexe D : Guide d'installation . . . 31

TABLE DES FIGURES

2.1 Diagramme de cas d'utilisation de CyberSentinel-LLM 9

3.1 Architecture globale de CyberSentinel-LLM 12
3.2 Diagramme de classes de CyberSentinel-LLM 14
3.3 Diagramme de sequence — flux d'une alerte jusqu'a l'explication LLM+RAG 15
3.4 Diagramme d'activite — traitement d'un log entrant 16

5.1 Architecture globale de CyberSentinel-LLM 28

LISTE DES TABLEAUX

1.1 Comparaison IDS par signatures vs IDS comportemental 5
1.2 Comparaison Isolation Forest vs Local Outlier Factor 5
1.3 Comparaison LLM sans RAG vs LLM avec RAG en cybersecurite 7

2.1 Exigences fonctionnelles de CyberSentinel-LLM 9

3.1 Description des composants backend 13

4.1 Endpoints principaux de l'API CyberSentinel-LLM 18
4.2 Features utilisees dans le pipeline ML 18
4.3 Niveaux d'action du module de defense applicative 21

5.1 Metriques d'evaluation du module de detection 24
5.2 Synthese des fonctionnalites — etat de l'implementation 25
5.3 Variables d'environnement de CyberSentinel-LLM 31

LISTE DES ABREVIATIONS

API : Application Programming Interface
ASR : Automatic Speech Recognition
ChromaDB : Base de donnees vectorielle locale open source
DB : Database (Base de Donnees)
GeoIP : Geographic Internet Protocol
HTTP : HyperText Transfer Protocol
IA : Intelligence Artificielle
IDS : Intrusion Detection System
IF : Isolation Forest
IPS : Intrusion Prevention System
JSON : JavaScript Object Notation
JWT : JSON Web Token
LLM : Large Language Model (Grand Modele de Langage)
LOF : Local Outlier Factor
ML : Machine Learning
MITRE ATT&CK : Adversarial Tactics, Techniques and Common Knowledge
MVP : Minimum Viable Product
NLP : Natural Language Processing
ORM : Object Relational Mapping
PFE : Projet de Fin d'Etudes
RAG : Retrieval-Augmented Generation (Generation Augmentee par Recuperation)
RBAC : Role-Based Access Control
REST : Representational State Transfer
SIEM : Security Information and Event Management
SOC : Security Operations Center
SSH : Secure Shell
SSE : Server-Sent Events
STT : Speech-to-Text (Parole vers Texte)
TIC : Technologies de l'Information et de la Communication
TTS : Text-to-Speech (Synthese vocale)
UI : User Interface
WAV : Waveform Audio File Format

INTRODUCTION GENERALE

Contexte et justification

La surface d'attaque des systemes d'information s'accroit rapidement, portee par l'heterogeneite des reseaux, l'essor des services numeriques et l'augmentation des usages mobiles. Au Tchad, la digitalisation est en pleine croissance : en 2025, le pays compte **2,74 millions d'internautes** (13,2% de la population) et **14,5 millions de connexions mobiles actives** (69,8%). Des projets structurants sont en cours, tels que le Data Center National en voie de certification, des projets de fibre optique vers le Niger, la Libye et l'Egypte, et l'objectif ambitieux d'atteindre 30% de penetration Internet d'ici 2030.

Cependant, cette digitalisation rapide s'accompagne de menaces croissantes. En 2024, une cyberattaque majeure orchestree par le groupe Anonymous Sudan a cible le principal operateur telecom tchadien, causant une coupure Internet totale. Les attaques SSH par force brute persistent regionalement sur les serveurs Linux, et les fraudes au Mobile Money explosent en Afrique centrale. Face a ces menaces, la maturite en cybersecurite du Tchad demeure limitee : peu d'outils SIEM ou SOC nationaux, dependance a des audits etrangers, et un manque criant d'experts locaux malgre les efforts de l'ANSICE (300 jeunes formes en 2024, 30 cadres en 2025).

Dans ce contexte, automatiser la detection et fournir une assistance explicative aux analystes SOC constituent des besoins prioritaires. C'est pour repondre a ce defi que le projet **CyberSentinel-LLM** a ete concu : une plateforme pedagogique qui combine detection d'anomalies par Machine Learning, assistance conversationnelle vocale par un modele de langage local augmente par RAG, et geolocalisation des sources d'attaque.

Objectif general

Concevoir et realiser une plateforme pedagogique, nommee **CyberSentinel-LLM**, qui combine detection d'anomalies (ML), assistance conversationnelle vocale par LLM local augmente par RAG, et geolocalisation des sources d'attaque, pour ameliorer l'efficacite et l'explicabilite du triage d'alertes au sein d'un Security Operations Center.

Objectifs specifiques

- ☞ Developper un pipeline d'ingestion de logs et de detection d'anomalies (IsolationForest / LOF + fallback heuristique) ;
- ☞ Implementer un module RAG base sur ChromaDB et MITRE ATT&CK pour enrichir les reponses du LLM ;
- ☞ Implementer une API backend (FastAPI) et une interface frontend (Next.js) avec streaming temps reel (SSE) ;
- ☞ Integrer un LLM local (Ollama / Llama 3.2) pour fournir des explications structurees d'incidents avec score de confiance pondere ;
- ☞ Integrer une interface vocale bidirectionnelle : transcription audio via Faster-Whisper (STT) et synthese vocale via Piper (TTS) ;
- ☞ Integrer la geolocalisation des IPs sources via GeoLite2 et visualiser les menaces sur une carte mondiale interactive ;
- ☞ Fournir des outils de labellisation humaine et des metriques d'evaluation (precision, recall, f1, roc_auc) ;
- ☞ Documenter et deployer un scenario de demonstration reproductible localement.

Question centrale de la recherche

Comment concevoir une plateforme cyberdefensive conversationnelle vocale exploitant le raisonnement contextuel d'un LLM augmente par RAG tout en maintenant une precision operationnelle, une robustesse et une confiance mesurable, dans des environnements de menace dynamiques et avec une architecture simple et reproductible dans le cadre d'un PFE ?

Hypotheses de la recherche

- ☞ **H1 :** Un pipeline minimaliste base sur des features simples orientees SSH permet de demontrer un fil rouge pedagogique complet ;
- ☞ **H2 :** Les algorithmes IsolationForest et LOF suffisent pour un MVP pedagogique combines a un fallback heuristique robuste ;
- ☞ **H3 :** Un LLM local augmente par RAG ameliore significativement la qualite des explications d'alertes par rapport a un LLM sans base de connaissances, tout en maintenant le principe *human-in-the-loop* ;
- ☞ **H4 :** L'integration d'une interface vocale (STT + TTS) enrichit l'experience de l'analyste SOC sans degrader les performances sur materiel contraint.

Positionnement du projet

CyberSentinel-LLM se positionne comme une **preuve de concept pedagogique** (niveau Licence) : une demonstration technique et experimentale destinee a valider la faisabilite d'un SOC augmente par LLM, RAG et interface vocale sur un perimetre restreint. Le projet ne pretend pas a une solution industrielle, mais pose les bases conceptuelles et techniques d'une telle approche dans le contexte tchadien.

CHAPITRE 1

ETAT DE L'ART ET GENERALITES

Introduction

La cybersecurite est devenue un enjeu strategique majeur pour les organisations a l'echelle mondiale. Ce chapitre presente un etat de l'art sur les systemes de detection d'intrusions, leurs limites, et l'apport du Machine Learning, des modeles de langage, des architectures RAG et des interfaces vocales pour les renforcer.

1.1 Presentation des IDS et IPS

Un **systeme de detection d'intrusions (IDS)** est un outil de surveillance qui analyse le trafic reseau ou les journaux systeme afin d'identifier des activites suspectes ou malveillantes. Un **systeme de prevention d'intrusions (IPS)** va plus loin en bloquant automatiquement les menaces detectees. On distingue deux grandes approches :

- **Approches par signatures :** comparaison du trafic observe avec une base de signatures d'attaques connues. Outils phares : *Snort* et *Suricata*. Tres efficaces contre les attaques connues, mais incapables de detecter les menaces inconnues (zero-day) ;
- **Approches comportementales / anomalies :** etablissement d'un profil de comportement normal du systeme, toute deviation significative constituant une alerte. Plus adaptees a la detection de nouvelles menaces, mais sujettes a un taux de faux positifs eleve.

1.2 Limites des approches classiques

Les IDS/IPS classiques presentent plusieurs limites importantes :

- **Volume d'alertes saturant :** les SOC recoivent des milliers d'alertes quotidiennes. Une etude recente montre que les analystes SOC passent en moyenne 44% de leur temps sur des faux positifs [5] ;

TABLE 1.1 – Comparaison IDS par signatures vs IDS comportemental

Critere	**Par signatures**	**Comportemental**
Detection menaces connues	Excellente	Moyenne
Detection zero-day	Nulle	Bonne
Taux de faux positifs	Faible	Eleve
Mise a jour requise	Frequente (base signatures)	Non
Exemples d'outils	Snort, Suricata	IsolationForest, LOF

- ☞ **Taux eleve de faux positifs :** epuisant les analystes et masquant les menaces reelles ;
- ☞ **Manque d'explicabilite :** les outils classiques signalent une anomalie sans en expliquer le contexte ;
- ☞ **Fragmentation contextuelle :** les alertes sont souvent isolees, sans correlation automatique.

1.3 Apport du Machine Learning en detection d'anomalies

Le Machine Learning permet d'apprendre automatiquement le comportement normal d'un systeme et de detecter les deviations significatives.

- ☞ **Isolation Forest (IF) :** algorithme base sur l'isolation aleatoire des points, propose par Liu et al. [1]. Les anomalies etant rares et distinctes, elles sont isolees plus rapidement que les points normaux ;
- ☞ **Local Outlier Factor (LOF) :** algorithme base sur la densite locale, introduit par Breunig et al. [2]. Un point est considere comme une anomalie si sa densite locale est significativement inferieure a celle de ses voisins.

Dans **CyberSentinel-LLM**, un mecanisme de fallback heuristique pallie automatiquement ces limitations lorsque le volume de donnees est insuffisant.

TABLE 1.2 – Comparaison Isolation Forest vs Local Outlier Factor

Critere	**Isolation Forest**	**Local Outlier Factor**
Type	Ensemble (arbres)	Densite locale
Complexite	Faible	Moyenne
Donnees requises	Peu (≥ 20)	Moyenne (≥ 50)
Interpretabilite	Faible	Faible
Sensibilite au bruit	Robuste	Sensible
Usage dans le projet	Score principal	Score secondaire

1.4 Apport des LLM et principe Human-in-the-Loop

Les **Grands Modeles de Langage (LLM)** ouvrent de nouvelles perspectives pour l'assistance aux analystes SOC [6] :

- ☞ **Reformuler et contextualiser** une alerte technique en langage naturel comprehensible ;
- ☞ **Synthetiser** plusieurs evenements correles pour reconstituer un scenario d'attaque plausible ;
- ☞ **Proposer des hypotheses et des actions** d'investigation non-executives, laissant la decision finale a l'analyste humain.

Le principe **Human-in-the-Loop** est fondamental : le LLM n'agit jamais de facon autonome sur le systeme. Des etudes recentes confirment que les LLM peuvent reduire significativement le temps de triage des alertes SOC [5].

Dans **CyberSentinel-LLM**, le modele *Llama 3.2* via **Ollama** est utilise localement, garantissant la confidentialite des donnees.

1.5 Retrieval-Augmented Generation (RAG) pour la cybersecurite

La **Generation Augmentee par Recuperation (RAG)**, introduite par Lewis et al. [3], enrichit les reponses d'un LLM en recuperant dynamiquement des informations pertinentes depuis une base de connaissances externe. Cette approche resout la tendance des LLM a *halluciner* lorsqu'ils manquent de contexte specifique.

Dans le domaine de la cybersecurite, le RAG presente des avantages importants [4] :

- ☞ **Ancrage factuel :** les recommandations sont fondees sur des techniques d'attaque reelles (MITRE ATT&CK) ;
- ☞ **Mise a jour sans reentrainement :** la base peut etre enrichie sans modifier les poids du LLM ;
- ☞ **Tracabilite :** chaque recommandation est associee a une technique MITRE precise.

Dans **CyberSentinel-LLM**, le RAG est implemente avec **ChromaDB** et le modele *all-MiniLM-L6-v2* (384 dimensions, CPU-only). La base indexe 15 techniques MITRE ATT&CK. Pour chaque incident, les 3 techniques les plus pertinentes sont recuperees par similarite cosinus.

1.6 Interface vocale : STT et TTS pour les SOC

L'integration d'interfaces vocales dans les outils d'analyse de securite vise a reduire la friction cognitive des analystes SOC.

- ☞ **Speech-to-Text (STT) : Faster-Whisper** (modele *base*, quantise int8, CPU-only), derive des travaux de Radford et al. [7], assure la transcription en francais. Ce modele offre un bon compromis precision/performance sur materiel contraint ;

TABLE 1.3 – Comparaison LLM sans RAG vs LLM avec RAG en cybersecurite

Critere	**LLM sans RAG**	**LLM avec RAG**
Source des reponses	Memoire d'entrainement	Base MITRE ATT&CK
Risque d'hallucination	Eleve	Faible
Tracabilite	Nulle	Technique MITRE citee
Mise a jour	Reentrainement requis	Mise a jour de la base
Usage dans le projet	Chat libre	Explication d'incidents

☞ **Text-to-Speech (TTS) : Piper** avec la voix neurale francaise *fr_FR-upmc-medium* est utilise pour la synthese vocale. En cas d'indisponibilite, **espeak-ng** assure un fallback immediat.

Cette combinaison STT + TTS transforme la console SOC en un **assistant conversationnel vocal** : l'analyste parle au micro, le systeme transcrit, le LLM analyse avec contexte RAG, et la reponse est lue a voix haute — entierement en local, sans API cloud.

Conclusion du Chapitre 1

Ce chapitre a presente l'evolution des systemes de detection d'intrusions, leurs limites et l'apport combine du Machine Learning, des LLM, des architectures RAG et des interfaces vocales. Ces elements justifient l'approche de **CyberSentinel-LLM**. Le chapitre suivant analysera les besoins fonctionnels et non fonctionnels du systeme.

CHAPITRE 2

ANALYSE DES BESOINS ET SPECIFICATIONS

Introduction

Ce chapitre presente l'analyse des besoins qui ont guide la conception de **CyberSentinel-LLM**. La plateforme doit repondre a des exigences precises tout en restant deployable localement sur une machine modeste (Intel Core i7 11^{e} generation, 16 GB RAM, sans GPU dedie).

2.1 Cas d'usage principaux

Les cas d'usage ont ete definis a partir des besoins operationnels d'un analyste SOC :

- **Detection automatique d'anomalies :** ingestion de logs reseau (SSH), scoring ML et generation automatique d'alertes ;
- **Triage et gestion des incidents :** regroupement d'alertes correlees en incidents, suivi du statut ;
- **Explication d'incidents via LLM + RAG :** analyse contextuelle ancree dans MITRE ATT&CK avec score de confiance pondere ;
- **Console SOC vocale :** dicter des requetes via micro (STT Faster-Whisper) et ecouter les reponses (TTS Piper) ;
- **Tableau de bord SOC :** KPI, timeline des alertes et cartographie GeoIP sur carte mondiale interactive ;
- **Labellisation et metriques :** validation humaine (true_attack / false_positive) et calcul precision/recall/f1/roc_auc ;
- **Defense applicative :** politique de reponse (allow / monitor / review / block_temp) avec approbation humaine.

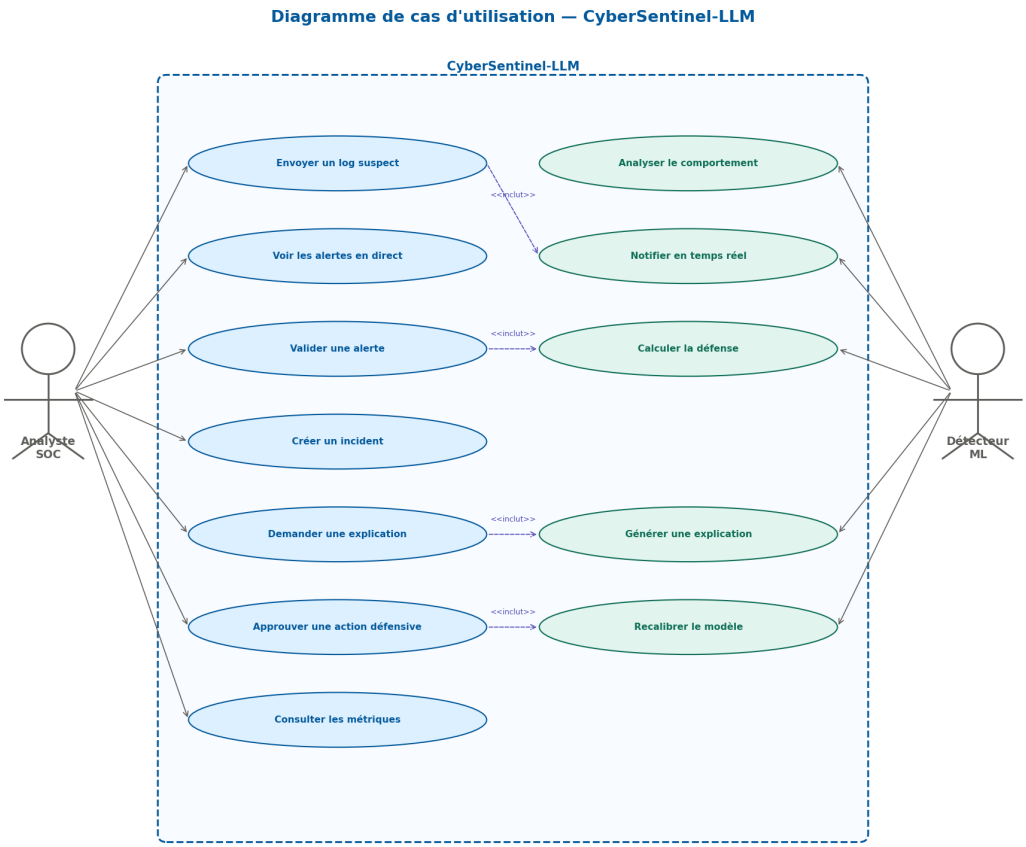


FIGURE 2.1 – Diagramme de cas d'utilisation de CyberSentinel-LLM

TABLE 2.1 – Exigences fonctionnelles de CyberSentinel-LLM

ID	**Fonctionnalite**	**Description**
EF01	Ingestion de logs	Recevoir et traiter 1..N logs via POST /ingest/logs
EF02	Scoring ML	Calculer un score d'anomalie 0..1 par log ingere
EF03	Gestion des alertes	Lister, filtrer et labelliser les alertes
EF04	Gestion des incidents	Creer, consulter et suivre les incidents
EF05	Explication LLM + RAG	Explication JSON ancree dans MITRE ATT&CK
EF06	Chat conversationnel	Interface de chat textuel et vocal avec le LLM
EF07	Streaming temps reel	Diffuser les alertes en temps reel via SSE
EF08	Cartographie GeoIP	Localisation des IPs sources sur Leaflet
EF09	Metriques ML	Calculer precision/recall/f1/roc_auc
EF10	Defense applicative	Actions de blocage avec approbation humaine
EF11	Score de confiance	Score pondere : ML (40%) + RAG (30%) + LLM (30%)
EF12	STT (transcription)	Transcrire la parole via Faster-Whisper
EF13	TTS (synthese vocale)	Lire les reponses via Piper ou espeak-ng

2.2 Exigences fonctionnelles

2.3 Exigences non fonctionnelles

- ☞ **Replicabilite locale :** deploiement 100% local (Ollama, SQLite, ChromaDB, Piper, via `start.sh`) sans cloud ;
- ☞ **Contrainte materielle :** CPU uniquement, 16 GB RAM, budget memoire total inferieur a 9 GB ;
- ☞ **Performance minimale :** usage pedagogique et demonstratif sur poste de developpement ;
- ☞ **Securite minimale :** CORS configurable, rate limiting par IP, security headers HTTP ;
- ☞ **Maintenabilite :** architecture modulaire (routers / services / models) ;
- ☞ **Principe Human-in-the-Loop :** le LLM ne prend aucune decision autonome ; toute action defensive necessite approbation humaine.

Conclusion du Chapitre 2

Ce chapitre a defini les cas d'usage et les exigences de **CyberSentinel-LLM**. Ces specifications guideront la conception architecturale et l'implementation presentees dans les chapitres suivants.

CHAPITRE 3

CONCEPTION LOGICIELLE ET ARCHITECTURE

Introduction

Ce chapitre presente l'architecture globale de **CyberSentinel-LLM**, la structure du code et les composants principaux du systeme.

3.1 Architecture globale

CyberSentinel-LLM repose sur une **architecture monolithique modulaire orientee API** comprenant six couches :

- ☞ **Couche 1 — Ingestion :** parsing des logs reseau, enrichissement GeoIP via GeoLite2 ;
- ☞ **Couche 2 — Detection ML :** IsolationForest, LOF, fallback heuristique, scoring 0..1 ;
- ☞ **Couche 3 — RAG + LLM :** ChromaDB + MITRE ATT&CK, Ollama / Llama 3.2 pour l'analyse des incidents ;
- ☞ **Couche 4 — Interface vocale :** Faster-Whisper (STT) et Piper (TTS), locaux sur CPU ;
- ☞ **Couche 5 — Backend API :** FastAPI modulaire, persistance SQLite via SQLModel/SQLAlchemy ;
- ☞ **Couche 6 — Frontend :** Next.js 14, interface SOC dark mode avec dashboard, alertes, incidents, demo, chat vocal et carte.

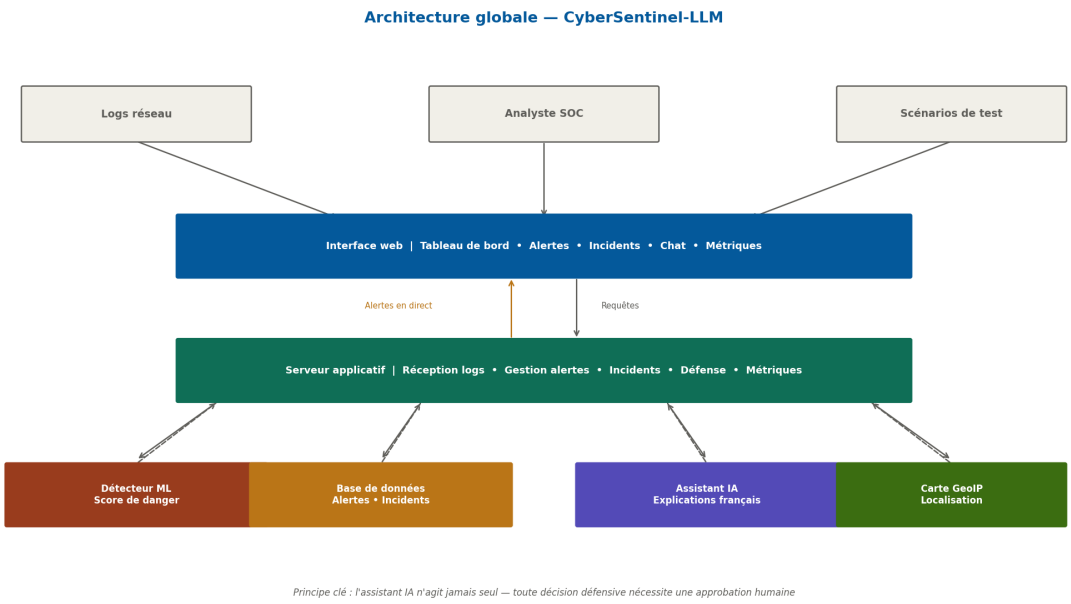


FIGURE 3.1 – Architecture globale de CyberSentinel-LLM

3.2 Arborescence du projet

Structure du projet CyberSentinel-LLM

```
cybersentinel-llm/
|-- backend/
|   |-- app/
|   |   |-- core/           # Configuration et settings
|   |   |-- db/             # Connexion base de donnees SQLite
|   |   |-- routers/        # Contrats API metier
|   |   |-- endpoints/      # Endpoints LLM avances
|   |   |   |-- llm_chat.py       # Chat + streaming SSE
|   |   |   |-- chat_store.py     # Persistance conversations
|   |   |   |-- assistant.py      # Assistant contextualise
|   |   |   |-- transcribe.py     # STT via Faster-Whisper
|   |   |   `-- tts.py            # TTS via Piper/espeak-ng
|   |   |-- models/         # Entites SQLModel
|   |   `-- services/       # Logique technique transverse
|   |       |-- detection.py      # Pipeline ML (IF/LOF)
|   |       |-- rag_service.py    # ChromaDB + MITRE ATT&CK
|   |       |-- llm_ollama.py     # LLM + RAG + confiance
|   |       |-- geoip.py          # GeoLite2 offline
|   |       |-- sse.py            # Server-Sent Events
|   |       |-- whisper_service.py  # Transcription audio
|   |       `-- tts_service.py      # Synthese vocale
|   `-- start.sh            # Lanceur unifie
|-- frontend/
|   |-- app/
|   |   |-- dashboard/  |-- alerts/  |-- incidents/
|   |   |-- demo/       |-- chat/    `-- model/
|   `-- components/
|       |-- WorldMap.tsx    # Carte Leaflet dark mode
|       `-- ui/             # Badge, Card, Button
`-- data/
```

3.3 Description des composants backend

TABLE 3.1 – Description des composants backend

Composant	Role
routers/ingest.py	Ingestion de logs, scoring ML, creation alertes, enrichissement GeoIP, publication SSE
routers/incidents.py	Creation et gestion des incidents
routers/defense.py	Politique de defense applicative
routers/metrics.py	Calcul precision/recall/f1/roc_auc
services/detection.py	Pipeline ML : IF/LOF, scoring, fallback
services/rag_service.py	ChromaDB + MITRE ATT&CK : indexation, retrieve_context(), get_rag_relevance_score()
services/llm_ollama.py	LLM + RAG : prompt enrichi, JSON force, retry, formule de confiance ponderee
services/whisper_service.py	Transcription audio WAV en francais (Faster-Whisper, base, int8, CPU)
services/tts_service.py	Synthese vocale francaise via Piper (fr_FR-upmc-medium) + fallback espeak-ng
endpoints/transcribe.py	POST /llm/transcribe : WAV -> texte
endpoints/tts.py	POST /llm/tts : texte -> WAV
services/geoip.py	Lookup GeoLite2 offline (pays, lat, lon)
services/sse.py	Queues asyncio et endpoint /stream/alerts
models/	Alert, Incident, LLMTrace, IPEnrichment, AlertLabel, Conversation, ChatMessage, DefenseAction

3.4 Description du frontend

Le frontend est developpe avec **Next.js 14** (App Router), interface SOC dark mode :

- ☞ **/dashboard :** KPI globaux, carte Leaflet mondiale (CartoDB dark matter), timeline des alertes ;
- ☞ **/alerts :** alertes temps reel, labellisation (true_attack / false_positive), creation d'incidents ;
- ☞ **/incidents/[id] :** detail d'un incident, explication LLM+RAG avec decision humaine ;
- ☞ **/demo :** simulation SSH en 5 etapes animees ;
- ☞ **/chat :** console SOC vocale avec micro (STT), lecture des reponses (TTS), streaming token par token ;
- ☞ **/model :** metriques ML.

3.5 Diagrammes UML

3.5.1 Diagramme de classes

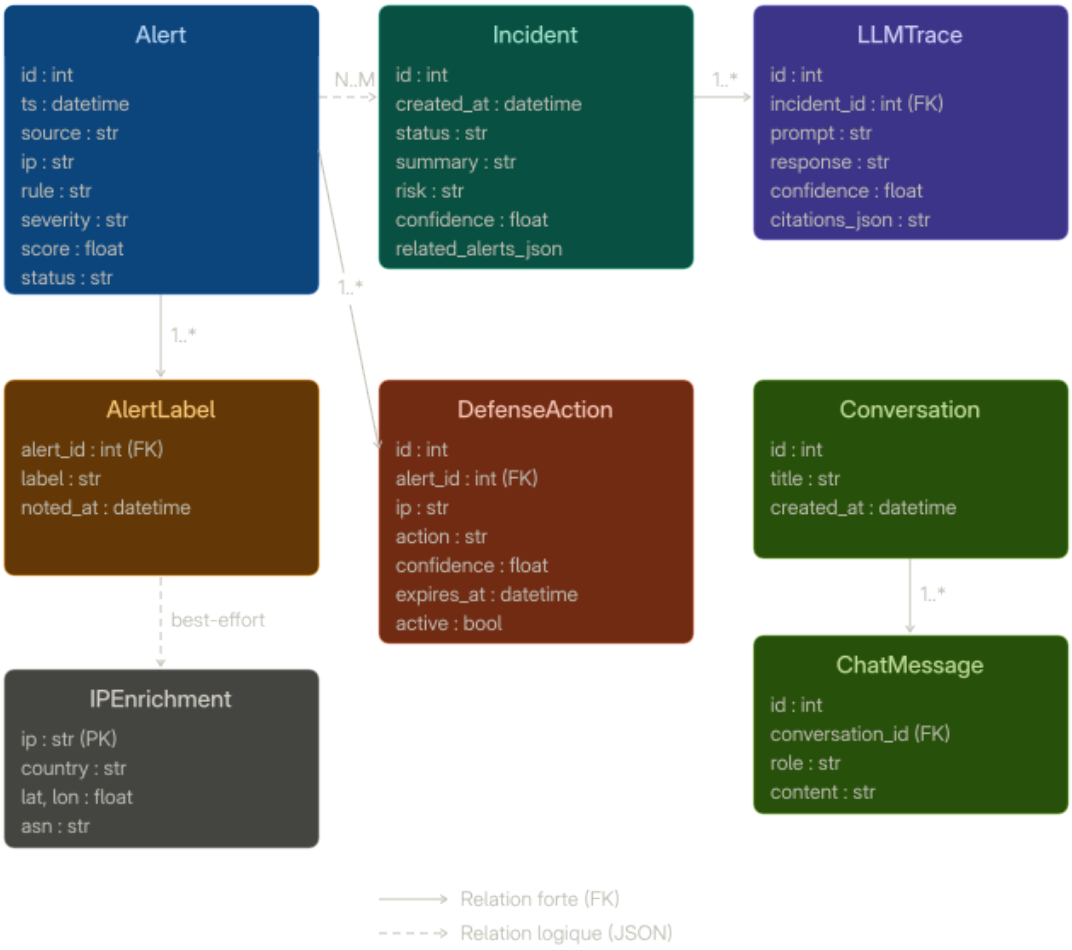


FIGURE 3.2 – Diagramme de classes de CyberSentinel-LLM

3.5.2 Diagramme de sequence

3.5.3 Diagramme d'activite

Conclusion du Chapitre 3

Ce chapitre a presente l'architecture globale en six couches et les composants de **CyberSentinel-LLM**. Le chapitre suivant detaillera l'implementation concrete de chaque composante.

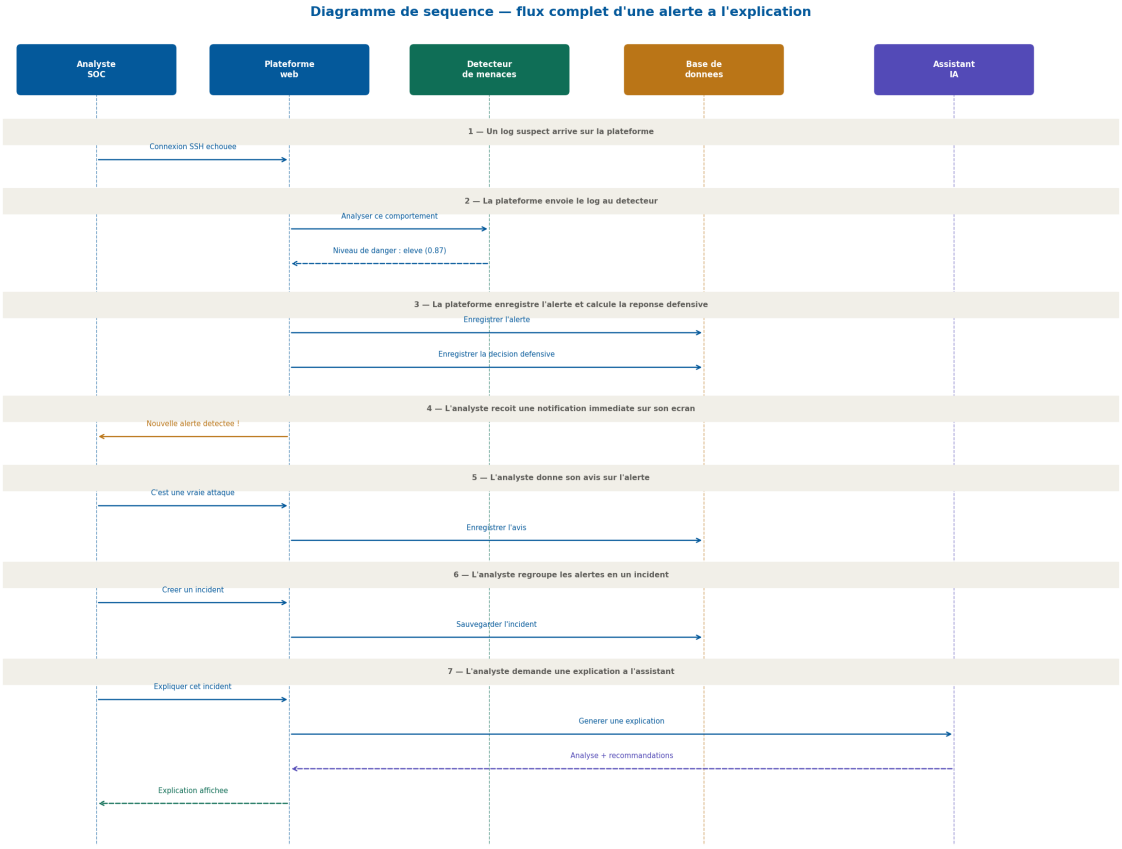


FIGURE 3.3 – Diagramme de sequence — flux d'une alerte jusqu'a l'explication LLM+RAG

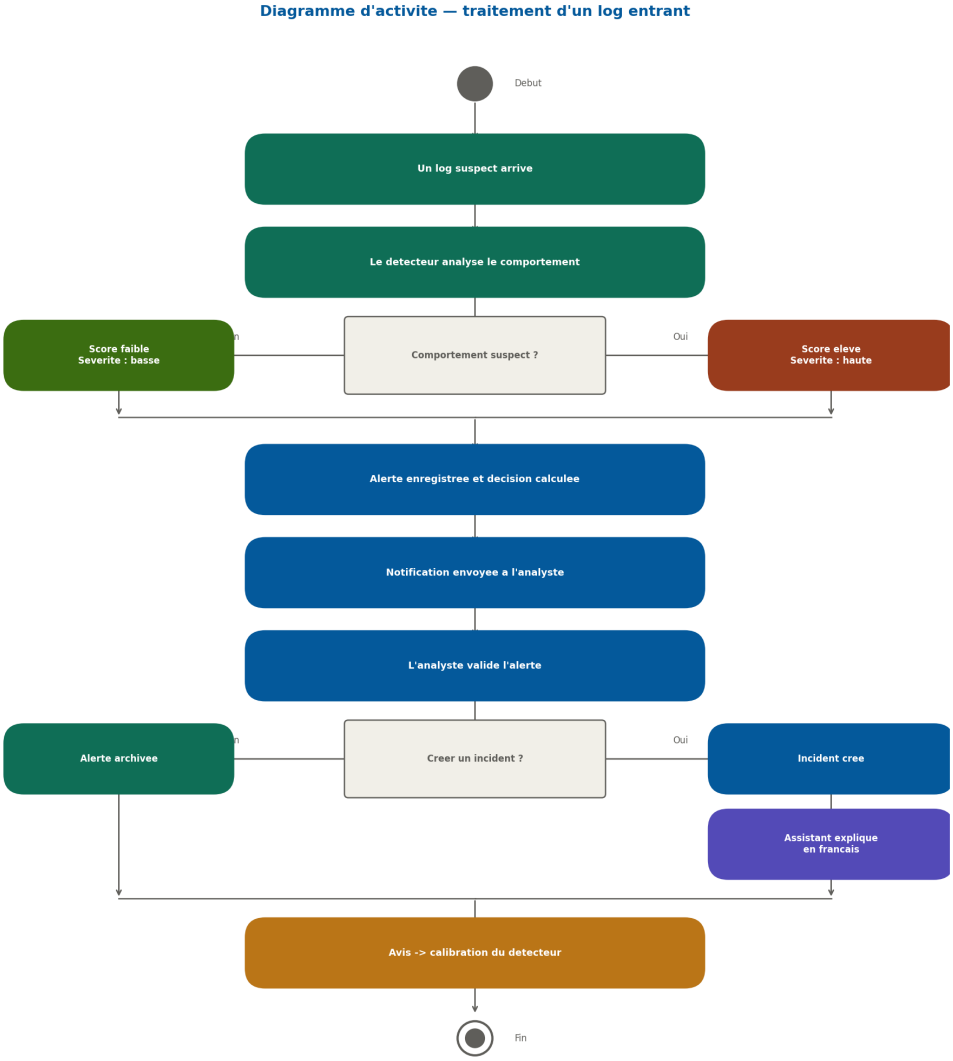


FIGURE 3.4 – Diagramme d’activite — traitement d’un log entrant

CHAPITRE 4

IMPLEMENTATION

Introduction

Ce chapitre presente les details d'implementation de **CyberSentinel-LLM** : endpoints API, pipeline ML, module RAG, module LLM, interface vocale STT/TTS, communication SSE et module de defense applicative.

4.1 Endpoints principaux

Exemple : POST /ingest/logs

```
// Requete
POST /ingest/logs
Content-Type: application/json
{
  "timestamp": "2025-06-15T14:32:00Z",
  "source": "sshd",
  "ip": "185.220.101.45",
  "user": "root",
  "event": "Failed password for root from 185.220.101.45"
}

// Reponse
{ "ingested": 1, "alert_id": 42, "score": 0.87, "severity": "high" }
```

4.2 Pipeline Machine Learning

Le pipeline ML suit les etapes suivantes :

1. **Extraction des features** : `[is_ssh, is_fail, hour_sin, hour_cos, ip_fail_count_60`
2. **Construction de la matrice d'entrainement** : `_build_training_matrix(N=200)` sur les 200 dernieres alertes ;

TABLE 4.1 – Endpoints principaux de l'API CyberSentinel-LLM

Methode	Endpoint	Role
GET	/health	Etat API + DB + Ollama
POST	/ingest/logs	Ingerer logs, scorer, creer alertes
GET	/alerts	Lister alertes (filtres status, severity)
POST	/incidents/from-alerts	Creer incident depuis liste d'alertes
POST	/llm/incidents/{id}/explain	Explication LLM+RAG d'un incident
POST	/llm/chat	Chat global (stream ou non)
POST	/llm/transcribe	Audio WAV vers texte (Faster-Whisper)
POST	/llm/tts	Texte vers audio WAV (Piper)
GET	/stream/alerts	Flux SSE temps reel
GET	/map/points	Points GeoIP enrichis
POST	/demo/generate	Generation scenarios SSH simules
POST	/demo/reset	Remise a zero de la base de demonstration
GET	/metrics/model	Metriques ML (precision/recall/f1)
POST	/defense/actions/{id}/approve	Approuver une action defensive
POST	/validation/alerts/{id}	Labelliser une alerte

3. **Scoring** via IF/LOF/hybride selon `DETECTOR` ;
4. **Application du seuil** `SCORE_THRESHOLD` (defaut : 0.6) ;
5. **Mapping score -> severite** : high (≥ 0.8), medium (≥ 0.5), low (< 0.5).

TABLE 4.2 – Features utilisees dans le pipeline ML

Feature	Description
`is_ssh`	Booleen : le log provient-il d'un service SSH ?
`is_fail`	Booleen : echec d'authentification ?
`hour_sin`	Encodage cyclique de l'heure (sinus)
`hour_cos`	Encodage cyclique de l'heure (cosinus)
`ip_fail_count_60m`	Nombre d'echecs de l'IP sur 60 minutes

Formule de fallback heuristique (< 20 samples)

$$score = 0.2 + 0.5 \times is_fail + \min(ip_fail_count, 5) \times 0.05$$

4.3 Module RAG — Retrieval-Augmented Generation

Le module RAG est implemente dans `services/rag_service.py` avec **ChromaDB** comme base vectorielle locale.

Initialisation et indexation

Le mode d'initialisation est controle par `RAG_INIT_MODE` (defaut : `lazy`). En mode lazy, le RAG s'initialise au premier appel. Il indexe 15 techniques MITRE ATT&CK avec *all-MiniLM-L6-v2* (384 dimensions, CPU, ~500 MB RAM). L'indexation est idempotente.

Score de pertinence RAG

La fonction `get_rag_relevance_score()` retourne un score de similarite cosinus normalise :

Conversion distance cosinus vers similarite

$$similarity = \max\left(0, \min\left(1, 1 - \frac{distance}{2}\right)\right) \quad \text{avec } distance \in [0, 2]$$

4.4 Module LLM avec score de confiance pondere

Prompt enrichi par RAG

Le prompt template injecte le contexte MITRE ATT&CK et force une reponse au format JSON strict :

Template de prompt RAG enrichi

```
Tu es un expert SOC specialise en cybersecurite.

Contexte MITRE ATT&CK recupere automatiquement :
{rag_context}

Incident a analyser : {incident_summary}
Alertes associees : {alert_ids}

Reponds UNIQUEMENT avec un objet JSON valide :
{
  "severity_score": <0-10>,
  "attack_type": "<nom attaque en francais>",
  "mitre_technique": "<T1110.001>",
  "mitre_name": "<nom technique MITRE>",
  "explanation": "<2-3 phrases en francais>",
  "recommended_actions": ["action1", "action2", "action3"],
  "confidence_score": <0.0-1.0>,
  "confidence_justification": "<pourquoi ce score>"
}
```

Formule de score de confiance pondere

$$confidence_{final} = (score_{ML} \times 0.4) + (score_{RAG} \times 0.3) + (score_{LLM} \times 0.3)$$

— $score_{ML}$: score d'anomalie IsolationForest/LOF (0..1)

— $score_{RAG}$: similarite cosinus MITRE ATT&CK (0..1)

— $score_{LLM}$: auto-evaluation du LLM (0..1, extrait du JSON)

Robustesse et fallback

Trois niveaux de degradation gracieuse :

1. **Appel normal :** prompt RAG enrichi, mode JSON force ;
2. **Retry :** prompt simplifie si le parsing JSON echoue ;
3. **Fallback statique :** reponse JSON predefined si Ollama est indisponible — le systeme ne crashe jamais.

4.5 Modules vocaux — STT et TTS

Speech-to-Text (Faster-Whisper)

`services/whisper_service.py` utilise **Faster-Whisper** (modele *base*, int8, CPU). Le chargement est differe au premier appel. L'endpoint `POST /llm/transcribe` recoit un fichier WAV et retourne le texte transcrit en francais.

Text-to-Speech (Piper + espeak-ng)

`services/tts_service.py` utilise **Piper** avec la voix *fr_FR-upmc-medium*. Le modele ONNX est mis en cache localement. En cas d'indisponibilite, **espeak-ng** assure un fallback immediat. L'endpoint `POST /llm/tts` retourne un fichier WAV lisible par le navigateur via l'API Web Audio.

Flux vocal complet dans la page /chat

1. L'analyste clique sur le micro — le navigateur capture l'audio ;
2. L'audio WAV est envoye a `POST /llm/transcribe` — Whisper retourne le texte ;
3. Le texte est soumis au LLM — la reponse arrive en streaming ;
4. Chaque phrase est envoyee a `POST /llm/tts` — Piper la synthetise et l'audio est joue automatiquement.

4.6 SSE et communication temps reel

La communication temps reel est assuree par **Server-Sent Events (SSE)**. Le backend maintient une liste de queues `asyncio.Queue` — une par client connecte. A chaque nouvelle alerte, l'evenement est publie via `publish_alert_created()` et diffuse a tous les abonnes.

Caracteristiques : gestion de backpressure (queue_max=256), message de connexion hello, implementation in-memory sans broker.

4.7 Module de defense applicative

TABLE 4.3 – Niveaux d'action du module de defense applicative

Action	Description
`allow`	Aucune action — activite normale
`monitor`	Surveillance renforcee — activite suspecte faible
`review`	Necessite revision humaine — activite suspecte moderee
`block_temp`	Blocage temporaire (TTL configurable) avec approbation humaine obligatoire

Toute action `block_temp` necessite une approbation explicite via `POST /defense/actions/{id}/a`
Le principe **Human-in-the-Loop** est maintenu a tous les niveaux.

Conclusion du Chapitre 4

Ce chapitre a detaille l'implementation de **CyberSentinel-LLM** : pipeline ML, module RAG, module LLM avec confiance ponderee, interface vocale STT/TTS, SSE et defense applicative. Le chapitre suivant presentera les resultats.

CHAPITRE 5

TESTS, VALIDATION ET RESULTATS

Introduction

Ce chapitre presente les scenarios de demonstration, les mecanismes d'evaluation et les resultats obtenus lors des tests de **CyberSentinel-LLM**.

5.1 Scenarios de demonstration

Un scenario reproductible valide le flux complet en 5 etapes :

1. **Generation des logs :** `POST /demo/generate` produit des logs SSH (tentatives echouees depuis 185.220.101.45) ;
2. **Detection ML :** IsolationForest/LOF genere des alertes avec score et niveau de severite ;
3. **Analyse LLM + RAG :** le LLM recupere les 3 techniques MITRE les plus pertinentes (ex : T1110.001) et genere une explication JSON ;
4. **Geolocalisation :** l'IP est affichee sur la carte Leaflet (ex : Noeud Tor, Allemagne) ;
5. **Score de confiance :** le score pondere ML+RAG+LLM est affiche.

Un scenario vocal est aussi disponible via `/chat` : l'analyste dicte une question, Whisper transcrit, le LLM repond avec contexte RAG, et Piper lit la reponse a voix haute.

5.2 Mecanismes d'evaluation

5.3 Resultats observes

☞ **Pipeline ML :** ingestion, scoring ML et SSE fonctionnent correctement. Fallback heuristique actif en dessous de 20 samples ;

TABLE 5.1 – Metriques d'evaluation du module de detection

Metrique	**Formule**	**Interpretation**
Precision	$\frac{VP}{VP+FP}$	Proportion d'alertes pertinentes
Rappel	$\frac{VP}{VP+FN}$	Proportion d'attaques detectees
F1-Score	$\frac{2\times P\times R}{P+R}$	Moyenne harmonique P/R
ROC-AUC	Aire sous la courbe ROC	Performance globale du classifieur
Latence LLM	Temps de reponse moyen	Performance temps reel
Score RAG	Similarite cosinus [0..1]	Pertinence du contexte MITRE

- ☞ **RAG :** ChromaDB en mode lazy, 15 techniques MITRE indexees, reponse en moins de 100ms sur CPU ;
- ☞ **LLM :** explications coherentes en JSON avec technique MITRE. Score de confiance pondere entre 0.4 et 0.8 ;
- ☞ **STT (Faster-Whisper) :** transcription francaise fonctionnelle. Latence 1–3 secondes pour 5–10 secondes d'audio ;
- ☞ **TTS (Piper) :** synthese vocale francaise naturelle. Fallback espeak-ng automatique si Piper indisponible ;
- ☞ **GeoIP :** enrichissement GeoLite2 offline et carte Leaflet dark mode fonctionnels ;
- ☞ **Defense applicative :** actions correctement calculees, journalisees et affichees sur le frontend.

5.4 Tests automatises

- ☞ **Tests backend (pytest) :** sante (`GET /health`), ingestion (`POST /ingest/logs`), cartographie (`GET /map/points`), chat LLM (`POST /llm/chat`) ;
- ☞ **Tests frontend :** build Next.js et lint ESLint sans erreurs bloquantes.

Conclusion du Chapitre 5

Les tests confirment que **CyberSentinel-LLM** remplit ses objectifs : pipeline ML + LLM augmente par RAG, interface vocale STT/TTS, sur materiel contraint. Les limites identifiees constituent des pistes d'amelioration futures.

TABLE 5.2 – Synthese des fonctionnalites — etat de l'implementation

Fonctionnalite	**Statut**	**Remarque**
Ingestion + scoring ML	✓Complet	IF + LOF + fallback
RAG + ChromaDB + MITRE ATT&CK	✓Complet	15 techniques, lazy
LLM + explication JSON	✓Complet	Retry + fallback statique
Score de confiance pondere	✓Complet	ML(40%)+RAG(30%)+LLM(30%)
STT via Faster-Whisper	✓Complet	Modele base, int8, CPU
TTS via Piper + espeak-ng	✓Complet	Voix fr_FR + fallback
SSE temps reel	✓Complet	In-memory, sans broker
GeoIP + Carte Leaflet dark	✓Complet	GeoLite2 offline
Chat vocal (STT+TTS)	✓Complet	Micro + lecture reponse
Labeling + metriques	✓Complet	Labels humains
Defense applicative	✓Complet	Human-in-the-loop
ML supervise multi-classes	× Partiel	Anomaly + heuristique
Authentification/RBAC	× Absent	Hors perimetre PFE
MLOps	× Absent	Retrain offline manuel

CONCLUSION GENERALE ET PERSPECTIVES

Ce memoire a presente la conception et l'implementation de **CyberSentinel-LLM**, une plateforme pedagogique combinant detection d'anomalies par ML, assistance conversationnelle vocale par LLM local augmente par RAG, et geolocalisation des sources d'attaque.

Le projet demontre la viabilite technique sur materiel contraint (CPU, 16 GB RAM) : pipeline d'ingestion de logs, moteur de scoring IsolationForest + LOF avec fallback heuristique, module RAG (ChromaDB, 15 techniques MITRE ATT&CK), interface vocale bidirectionnelle (Faster-Whisper STT, Piper TTS), interface Next.js avec streaming SSE et carte geolocalisation, et LLM local (Ollama/Llama 3.2) avec score de confiance pondere. Le principe **Human-in-the-Loop** a ete maintenu tout au long du systeme.

Dans le contexte tchadien ou la cybersecurite est en construction, cette plateforme est accessible localement, independante de services cloud, adaptee aux equipes SOC en formation, et ancree dans MITRE ATT&CK.

Limites identifiees : architecture monolithique locale, features ML limitees au domaine SSH, absence d'authentification, latence LLM sur CPU (20–30 secondes), qualite vocale Piper inferieure aux solutions cloud.

Perspectives :

- ☞ Extension du corpus RAG avec CVE et Sigma rules ;
- ☞ Entrainement supervise multi-classes sur NSL-KDD/CICIDS2017 ;
- ☞ Amelioration pipeline vocal (modele Whisper small/medium) ;
- ☞ Separation en microservices + broker evenementiel (Redis/Kafka) ;
- ☞ Authentification et gestion des roles (RBAC/JWT) ;
- ☞ Pipeline MLOps complet (versioning, drift detection, CI/CD) ;
- ☞ Adaptation au contexte tchadien : formation des analystes locaux et integration avec les initiatives de l'ANSICE.

BIBLIOGRAPHIE

- **[1] Liu, F. T., Ting, K. M., Zhou, Z.-H.** (2008). Isolation Forest. *IEEE ICDM*, pp. 413–422.
- **[2] Breunig, M. M., Kriegel, H.-P., Ng, R. T., Sander, J.** (2000). LOF : Identifying Density-Based Local Outliers. *ACM SIGMOD Record*, 29(2), pp. 93–104.
- **[3] Lewis, P., Perez, E., Piktus, A. et al.** (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, vol. 33, pp. 9459–9474.
- **[4] Ferrag, M. A., Ndhlovu, M., Tihanyi, N. et al.** (2023). Revolutionizing Cyber Threat Detection with Large Language Models. *IEEE Access*, vol. 12, pp. 977–996.
- **[5] Motlagh, F. N., Hajizadeh, M., Majd, M. et al.** (2024). Large Language Models in Cybersecurity : State-of-the-Art. *arXiv :2402.00891*.
- **[6] Liu, Y., Ning, Z., Li, M. et al.** (2024). Towards Explainable Network Intrusion Detection using Large Language Models. *arXiv :2408.04342*.
- **[7] Radford, A., Kim, J. W., Xu, T. et al.** (2023). Robust Speech Recognition via Large-Scale Weak Supervision. *ICML*, pp. 28492–28518.
- **Scikit-learn** — *https ://scikit-learn.org/stable/*
- **FastAPI** — *https ://fastapi.tiangolo.com/*
- **Ollama** — *https ://ollama.com/*
- **ChromaDB** — *https ://docs.trychroma.com/*
- **MITRE ATT&CK** — *https ://attack.mitre.org/*
- **Faster-Whisper** — *https ://github.com/SYSTRAN/faster-whisper*
- **Piper TTS** — *https ://github.com/rhasspy/piper*
- **Next.js 14** — *https ://nextjs.org/docs*
- **DataReportal** (2025). Digital 2025 : Chad. *https ://datareportal.com/reports/digital-2025-chad*
- **ANSICE** — *https ://www.ansice.td/*

ANNEXES

Annexe A : Architecture globale du systeme

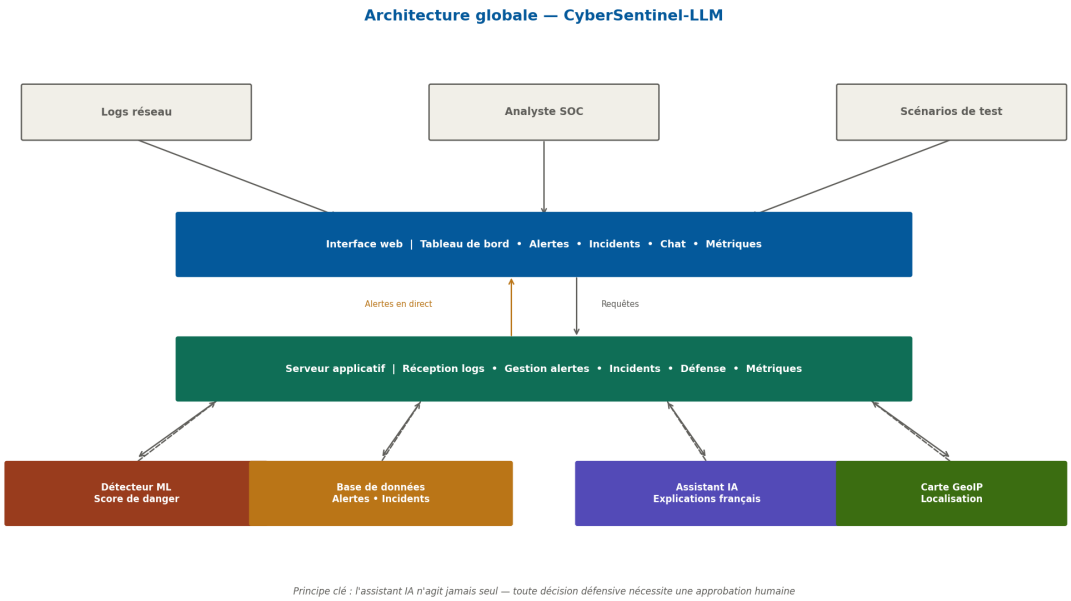


FIGURE 5.1 – Architecture globale de CyberSentinel-LLM

Annexe B : Exemples de payloads API

POST /ingest/logs — requete et reponse LLM structuree :

```
// Requete
{
  "timestamp": "2025-06-15T14:32:00Z",
  "source": "sshd",
  "ip": "185.220.101.45",
  "user": "root",
  "event": "Failed password for root from 185.220.101.45 port 22"
}

// Reponse LLM structuree (extrait)
{
  "severity_score": 8,
  "attack_type": "Force brute SSH",
  "mitre_technique": "T1110.001",
  "mitre_name": "Password Guessing",
  "explanation": "Tentatives repetees depuis IP Tor.
    Technique T1110.001 MITRE ATT&CK identifiee.",
  "recommended_actions": [
    "Bloquer l'IP via fail2ban",
    "Verifier auth.log des 24 dernieres heures",
    "Activer authentification par cle SSH"
  ],
  "confidence_score": 0.72,
  "confidence_justification": "ML=0.87, RAG=0.89, LLM=0.65"
}
```

Annexe C : Scripts utilitaires

gen_ssh_logs.py — Generateur de logs SSH simules

```
import argparse, requests, datetime, random

def generate_log(ip, user="root"):
    return {
        "timestamp": datetime.datetime.utcnow().isoformat() + "Z",
        "source": "sshd",
        "ip": ip,
        "user": user,
        "event": f"Failed password for {user} from {ip} port 22"
    }
# ======> EXTRAITS -- voir code source complet
```

retrain_offline.py — Recalibration du seuil ML

```
# Calcule le meilleur SCORE_THRESHOLD via maximisation F1-score
# Exporte model_config.json avec threshold optimal
# ======> EXTRAITS -- voir code source complet
```

Annexe D : Guide d'installation et d'execution

Prerequis :

- ☞ Python 3.10+ avec pip et virtualenv
- ☞ Node.js 18+ avec npm
- ☞ Ollama + modele llama3.2 (`ollama pull llama3.2`)
- ☞ GeoLite2-City.mmdb dans `backend/data/`
- ☞ espeak-ng : `sudo dnf install espeak-ng` (Fedora)

Lancement du backend

```
cd backend
source .venv/bin/activate
pip install -r requirements.txt
bash start.sh
# API : http://localhost:8000
# Swagger : http://localhost:8000/docs
```

Lancement du frontend

```
cd frontend
npm install
npm run dev
# Interface : http://localhost:3000
```

TABLE 5.3 – Variables d'environnement de CyberSentinel-LLM

Variable	Defaut	Description
OLLAMA_BASE_URL	localhost :11434	URL du serveur Ollama
OLLAMA_MODEL	llama3.2	Modele LLM utilise
SCORE_THRESHOLD	0.6	Seuil de detection ML
DETECTOR	hybrid	IF / LOF / hybrid
RAG_ENABLED	true	Activer/desactiver le RAG
RAG_INIT_MODE	lazy	off / background / lazy
DEFENSE_MODE	monitor	Mode defense par defaut
GEOLITE2_CITY_DB	auto-detecte	Chemin base GeoIP
APP_ENV	dev	dev ou final